SESSION 4 · PHASE 4

Monitoring, Observability & LLM Evaluation

How to know your model is still doing its job — after it ships.



Hands-on · Incidents · Interview prep

The MLOps Practitioner · MLOps MENA Community

ROADMAP

What we cover today

Two chapters go deep — Langfuse and RAGAS. The tools chapter gives you what you need and moves on.

01

Why models degrade

The four classic kinds of drift, plus the eight that only appear in RAG, LLM and agent systems.

02



The stack in one pass

Prometheus, Grafana and Evidently — what each is for, and the two or three features you will use.

03

What actually goes wrong

Four documented incidents — banking, computer vision, RAG and forecasting — where a model quietly stopped working and no dashboard noticed.

04

Langfuse

Langfuse in depth

The four screens: tracing, prompt management, scores, and token and cost usage.

05

RAGAS

RAGAS in depth

How each metric is actually computed, what it blames, and how to trust the number.

06

Interview preparation

Twelve questions you will be asked, what each is testing, and a strong answer.

LEARNING OBJECTIVES

By the end of this session you can…

1. **Explain why models degrade**
   Tell the four kinds of drift apart and give a real example of each.

2. **Instrument and alert**
   Expose metrics from a service, build a dashboard, and write alerts worth waking up for.

3. **Investigate an incident**
   Work through symptom, evidence, root cause and fix — in the right order, with the model last.

4. **Observe an LLM app end to end**
   Trace it, version its prompts, price it and score it with Langfuse.

5. **Measure answer quality**
   Know how each RAGAS metric is computed, what it blames, and when not to trust it.

WHERE THIS FITS

Phase 1
Packaging, FastAPI, Docker

Phase 2
MLflow, W&B, experiments

Phase 3
CI/CD, serving, Triton, vLLM

Phase 4
Monitoring — you are here

Phase 5
Optimization & edge deployment

Phases 1–3 get a model into production once. Phase 4 is what keeps it working.

CHAPTER 1

Why Models Degrade

- Models fail silently
- The four kinds of drift
- Where drift comes from
- What to watch without labels
- Picking a statistical test

THE PROBLEM

Software fails loudly. Models fail quietly.

A NORMAL OUTAGE

`500 Internal Server Error`

The pager fires in 30 seconds. The error rate spikes on the dashboard. Everyone knows within a minute.

Detected in: seconds

A MODEL FAILURE

`200 OK`

Every request succeeds. Latency is fine. The service is green. The predictions are wrong, and nothing in your stack knows it.

Detected in: weeks

91%

of ML models lose accuracy over time if nobody retrains them

~2 weeks

typical delay before someone notices, if you wait for business metrics

0

alerts your normal monitoring tool fires when accuracy quietly drops 15 points

Monitoring is not an extra. It is the only thing that tells you a deployed model has stopped working.

DRIFT TAXONOMY

Four things can shift — and they are not the same

X = your input features. Y = the true answer. $\hat{Y}$ = what the model predicted.

Data drift

the inputs change

You start seeing inputs that were rare in training. The rule connecting input to answer is unchanged.

EXAMPLE

A fraud model trained on Egyptian traffic now sees 40% of requests from Saudi Arabia.

Concept drift

the rule changes

The same input should now give a different answer. The dangerous one — your features can look perfectly normal.

EXAMPLE

"Three logins from new devices" meant fraud last year. After a UX change it means normal onboarding.

Prediction drift

the outputs change

The model's own output distribution moves. Your best early warning when labels arrive late or never.

EXAMPLE

Average predicted churn climbs from 0.12 to 0.31 over six weeks.

Label drift

the answers change

The mix of true answers shifts. Thresholds tuned on the old mix are now wrong.

EXAMPLE

Default rate goes from 2% to 7%. Your 0.5 cut-off no longer makes sense.

The trap: data drift is easy to measure and often harmless. Concept drift is hard to measure and always harmful.

ROOT CAUSES

Where drift actually comes from

Before you blame the model, check these four in order.

A — A bug upstream

A schema change, a unit change (cents → dollars), a new default value, an API that started returning nulls. Most "drift" is this.

Check: null rates and schema tests first

B — Season and cycles

Ramadan, Black Friday, end of quarter, school holidays. Real shifts that are not errors — do not retrain on them.

Check: compare to the same period last year

C — Your users changed

A new market, new pricing, a viral campaign, a competitor launch. The population you serve is genuinely different now.

Check: split the metric by region or cohort

D — Training ≠ serving

Preprocessing in the training pipeline does not match the serving path. Not drift at all — a bug that looks like drift.

Check: re-score logged serving features offline

Rule: rule out a broken pipeline before you ever schedule a retrain.

STRATEGY

Labels arrive too late to be your alarm

You find out whether a prediction was right days later, or never. So you alert on proxies now — and use labels, when they come, to check the proxies were telling the truth.

WATCH NOW

`seconds · no labels needed`

Is the service doing its job at all?

- Latency p50 / p95 / p99
- Requests per second, error rate
- Schema and type validation failures
- Missing-value rate per feature

This is your pager.

WATCH TODAY

`hours · no labels needed`

Do the inputs and outputs still look normal?

- Feature drift vs a frozen reference
- Mean predicted score, score histogram
- Positive rate — share above threshold
- Share of predictions near the threshold

This is your early warning.

CHECK LATER

`days or weeks · needs labels`

Were we actually right?

- True accuracy, F1, AUC, mAP
- Performance per segment
- Business KPI: conversion, loss, downtime

This is your audit — not your detector.

The proxies are your alarm.
The labels are your audit.

Accuracy is the only thing you actually care about — and it is useless as a detector, because by the time it moves the damage is already done. So you alert on the two left-hand columns. When labels finally arrive, you ask one question of them: did the proxies see it coming? If yes, trust them. If not, you are missing a signal.

CHOOSING A TEST

How do you prove a distribution moved?

Three tests cover almost everything you will meet. Learn these before the exotic ones.

TEST	DATA TYPE	WHAT IT GIVES YOU	USE IT FOR
Kolmogorov–Smirnov	Numeric	A statistic + a p-value	Continuous features: amount, age, latency, temperature
Chi-square	Categorical	A statistic + a p-value	Categories and class balance: country, channel, device
Jensen–Shannon	Both	One number between 0 and 1	Comparing drift across many features on the same scale

The mistake everyone makes

Reporting a p-value on its own. With 100,000 rows in a window, a test will flag a shift so small you would never care about it. Always report the size of the shift next to the p-value, and set your alert on the size.

Worth knowing exists

Wasserstein distance gives you the shift in real units ("moved by 40 EGP"). MMD and a domain classifier catch drift in the combination of features that per-feature tests miss — useful for embeddings. You do not need them on day one.

DRIFT TAXONOMY · RAG

A RAG system has four more ways to drift

The classic four still apply. These sit on top of them, and none show up in a normal drift report.

Embedding drift

the query and the index stop agreeing

The index was built with one embedding model and queries are encoded with another — after a rebuild, a library upgrade, or a provider silently updating a hosted embedding endpoint. The vectors no longer live in the same space.

WHAT DETECTS IT

`zero-hit rate · mean top-1 similarity · embedding model name asserted at startup`

Corpus drift

the documents move under you

Policies get rewritten, products are discontinued, pages are deleted. Deleted content stays retrievable if the index is not rebuilt, so the model confidently answers from a policy that no longer exists.

WHAT DETECTS IT

`age of retrieved documents · share of answers citing stale docs · index vs source doc count`

Retrieval drift

what comes back changes shape

The index is fine and the encoder is fine, but the distribution of what gets retrieved shifts — a few chunks start dominating every answer, or similarity scores slowly fall as the corpus grows.

WHAT DETECTS IT

`mean top-k similarity · chunk reuse concentration · context precision by week`

Query intent drift

users start asking different things

A product launch, a campaign or a news event changes what people ask. Your corpus never covered it, so retrieval returns near-misses and the model fills the gap by inventing.

WHAT DETECTS IT

`refusal rate · out-of-scope rate · weekly clustering of questions`

Our own bilingual case in chapter 3 was embedding drift and corpus drift at the same time — and it showed up in only one language.

DRIFT TAXONOMY · LLM & AGENTS

Four more that catch teams by surprise

These are the ones people do not think to instrument, because in classic ML they have no equivalent.

Provider drift

the model changed and you did not

The same model name points at a new snapshot. Behaviour shifts with no deploy on your side, no error, and nothing in your changelog to blame.

Pin a dated snapshot, never a floating alias. Run your golden dataset nightly, and treat a score change with no release as an incident.

Prompt drift

a hundred small edits

Nobody rewrote the prompt. Fifteen people tweaked it over six months, each change reasonable, and the accumulated result is a prompt nobody would have approved as a whole.

Version every prompt, link it to traces, and review the diff between the current production version and the one from six months ago.

Trajectory drift

agents quietly taking the long way

The agent still completes the task, so nothing fails. But average steps per task creeps from 4 to 9, tool errors rise, and cost per task doubles while the success rate looks unchanged.

Steps per task · tool call success rate · share of tasks hitting max iterations · cost per completed task.

Evaluator drift

your ruler changed length

The judge model gets upgraded, or its temperature is not pinned. Every score shifts and you spend a day investigating a quality problem that does not exist.

Pin the judge and its version, keep temperature at 0, re-run the calibration set, and never compare scores across judge versions.

Also worth instrumenting once you have the basics: guardrail block rate, context length growth, tokenizer or chunking changes, and refusal rate.

CHAPTER 2

The Stack, In One Pass



What each tool is for

Two or three features each

Enough to work with

Not a tour of every setting

GROUND RULES

Five tools, one job each

We are covering what each tool is for and the two or three features you will actually use. The documentation is very good — memorising it is not the skill.

	**Prometheus**	*Collect and store numbers over time*	Pulls metrics from your services on a timer · Stores them as labelled time series · PromQL to query and to define alerts
	**Grafana**	*Look at those numbers, and alert on them*	Dashboards over several data sources at once · Variables and deploy annotations · Dashboards provisioned as code
Evidently	**Evidently AI**	*Compare this week to training*	Drift reports over a reference dataset · Pass/fail tests you can run in CI · Snapshots you store and trend
Langfuse	**Langfuse**	*See inside an LLM application*	Traces of every step of a request · Prompt versioning and rollback · Scores, datasets and experiments
RAGAS	**RAGAS**	*Put a number on answer quality*	Reference-free metrics for live traffic · Splits blame: retrieval vs generation · Runs in CI as a quality gate

Everything after this chapter assumes you have these five straight. Langfuse and RAGAS get a full chapter each — the other three get what they need.

PROMETHEUS

Three things to understand, then move on



01 **It pulls, you publish**

Your service exposes a text page at /metrics. Prometheus reads it every 15 seconds. Your app never needs to know where Prometheus lives — and a target that stops answering is itself a signal you can alert on.

02 **Everything is a labelled time series**

A metric name plus labels. Counters only go up (query with rate()), gauges go up and down, histograms count observations per bucket. Labels are how you filter later — never put user_id or a filename in one.

03 **Exporters give you most of it free**

node_exporter for the machine, cAdvisor for containers, DCGM for NVIDIA GPUs, and one built into Triton and vLLM. You write metrics only for things unique to your model.

The one mistake that hurts: a label with unbounded values — user_id, request_id, a filename — creates a new time series per value and will eventually take Prometheus down, exactly when you need it most.

the promql you need

```
# a metric line
ml_predictions_total{
  version="3.2.1",
  outcome="defect"}  1843

# rps by version
sum(rate(
  ml_predictions_total[5m]
)) by (version)

# p95 latency
histogram_quantile(0.95,
  sum(rate(
   ml_latency_bucket[5m]
  )) by (le))

# is it alive?
up{job="model-api"} == 0
```

HANDS-ON

Instrumenting your service

Four metrics is a realistic starting point. Add more only when you find yourself wanting them.

src/api/main.py

```python
from prometheus_client import (Counter, Histogram,
                               Gauge, make_asgi_app)
PREDICTIONS = Counter("ml_predictions_total", "Served",
                      ["model", "version", "outcome"])   # rate()
LATENCY = Histogram("ml_inference_latency_seconds", "Latency",
                    ["model", "stage"], buckets=(.01, .05, .1, .5, 1.))
SCORE = Histogram("ml_prediction_score", "Output dist",
                  ["model"], buckets=[i/20 for i in range(21)])
INFLIGHT = Gauge("ml_inflight_requests", "In flight", ["model"])

app.mount("/metrics", make_asgi_app())
@app.post("/predict")
async def predict(req: Item):
    INFLIGHT.labels(MODEL).inc()
    try:
        score = float(model(preprocess(req)))
        SCORE.labels(MODEL).observe(score)
        return {"score": score}
    finally:
        INFLIGHT.labels(MODEL).dec()
```

PREDICTIONS

Throughput and the positive rate — your cheapest drift signal, and the one that would have caught Incident 01.

LATENCY by stage

Split preprocess, infer and postprocess. "The model is slow" is usually preprocessing.

SCORE

The output distribution. Prediction drift needs no labels and moves before anything else.

INFLIGHT

Saturation. This is what users feel, and it is not the same thing as utilization.

Running multiple workers? Set PROMETHEUS_MULTIPROC_DIR, or each worker reports only its own slice.

GRAFANA & ALERTING

One dashboard, four rows, and alerts worth waking up for

ROW 1 · Service — requests/sec · p50, p95, p99 latency · error rate · up{}

ROW 2 · Resources — GPU utilization, memory, temperature, clock · CPU · RAM · disk

ROW 3 · Model behaviour — score distribution · positive rate · validation failures · batch size

ROW 4 · Quality & drift — drifted features (30d) · accuracy on late labels · retrain history

Variables

A $model dropdown at the top beats twelve near-identical dashboards nobody maintains.

Annotations

A vertical line on every deploy and retrain. Half of all drift investigations end the moment you see the spike land on a release.

Provisioned as code

A dashboard built by clicking dies with the pod. Commit the JSON and ship it with the model.

ml_alerts.yml

```yaml
- alert: ModelTargetDown
  expr: up{job="api"} == 0
  for: 2m
  labels: {severity: page}
  annotations:
    runbook: "wiki/model-down"

- alert: NoPredictions
  expr: sum(rate(
    ml_predictions_total[10m]
  )) == 0
  for: 10m
```

Page vs warn

PAGE on symptoms users feel — errors, latency, no predictions, target down.

WARN on causes — drift, memory creeping, a growing queue. Alertmanager routes by severity label.

EVIDENTLY & THE METRICS STORE

The slow half: batch drift, stored and trended

Evidently

Prometheus answers "what is happening now". Evidently answers "how does this week compare to training".

Reports

Compare a current dataset to a frozen reference. Presets pick the right statistical test per column for you.

Tests

Pass/fail conditions on top of a report — "no more than 2 drifted columns". This is what runs in CI.

Snapshots

Every run serialises to timestamped JSON. This is the thing you save, so you can see a trend later.

jobs/daily_drift.py

```
report = Report([DataDriftPreset(),
                 DriftedColumnsCount(tests=[lte(2)])],
                include_tests=True)
snapshot = report.run(current_data=cur, reference_data=ref)
```

Store the result, always

One row per window, per column, per metric, in PostgreSQL — with the model version and which reference set you compared against. A drift number you throw away every night cannot show you a trend, and the trend is the whole point.

BEFORE A DRIFT SIGNAL BECOMES A RETRAIN, IT PASSES FOUR GATES

Sustained

Two days, not one bad window.

Not a bug

Nulls, schema and upstream jobs verified healthy first.

Enough labels

No new labels means you would be fitting noise.

Cooldown

No retrain within 7 days, and the challenger must beat the champion.

HEADS-UP

Metrics, logs and traces do different jobs

You will need all three eventually. Start with metrics; know when to reach for the others.

Metrics

Something changed

Numbers over time. Cheap and easy to alert on — as long as you keep label combinations under control.



Logs

What exactly happened

One detailed record per event. Expensive to index, unbeatable when you need to know what happened to request a41f at 14:32.



Loki

Traces

Where the time went

One request's journey across steps, with timing for each. The map of a distributed call — and of an LLM chain.



Langfuse

When ELK is the right answer

Fluent Bit ships, Elasticsearch indexes, Kibana searches. Choose it when you need full-text search across services, on-prem storage, or you already run Elastic for security.

One rule for ML: never sample your prediction logs

When logging gets expensive teams start sampling. But the rare class, the near-threshold case and the one segment that broke are exactly what sampling throws away. Sample the boring HTTP access logs instead.

CHAPTER 3

What Actually Goes Wrong

- Four documented incidents
- Banking · vision · RAG · forecasting
- How each one was found
- Why nobody noticed for weeks
- What changed afterwards

HOW TO READ THIS CHAPTER

Every incident has the same five parts

Not because it is a nice format — because this is the shape of every postmortem you will ever write.

Symptom	What someone noticed, and how long after it started. The gap between "it broke" and "we knew" is the number your monitoring is graded on.
What was found	The evidence people actually opened, in order. This is the part juniors skip and interviewers ask about.
Root cause	The real one, not the first plausible one. In this chapter the model is never the root cause. Not once in four.
The fix	Two fixes every time: stop the bleeding now, and make this class of failure impossible or loud next time.
The lesson	The instrument you were missing. If the fix does not add a signal, you will meet the same incident again.

Blameless — other companies too

We are not here to score points off four organisations that shipped something hard. Two of them published their own error rates, one submitted to an independent audit, and all four operated at a scale most of us never will.

If a postmortem ends with a person's name, the system stays broken and the next person hides the outage. If it ends with a missing alert, a missing contract test or a missing gate, you actually got safer.

Ask "what made this easy to do and hard to see?" — not "who did it?"

All four are public — audits, investigations and published measurements. Sources are on the last slide of this chapter.

INCIDENT 01

The amounts arrived in the wrong currency unit

Fraud detection · found 9 days late · zero alerts fired · the service was green the entire time

THE SYMPTOM

The fraud team asked why flagged transactions had "gone quiet". The flag rate had fallen from 3.1% to 0.4% overnight and stayed there for nine days.

No alert. No error. Latency normal. Every response 200 OK.

→

WHAT WE FOUND

No deploy in the window. Service metrics all flat.

The nightly drift job told the whole story: KS on transaction_amount was 0.87, and the mean had fallen by almost exactly 100×. Nobody had been reading that report.

→

ROOT CAUSE

An upstream team migrated the payments API and started sending amounts in piastres instead of pounds.

Every amount became 100× smaller, so nothing ever crossed the large-transaction thresholds the model had learned.

→

THE FIX

Immediate: multiply by 100 at the boundary, re-score the nine days.

Permanent: range assertions on every numeric feature at serving time, an alert on prediction rate, and a contract test in the upstream team's CI.

THE LESSON

The model was perfect. The data was wrong. Most "model degradation" is an upstream change nobody told you about — which is why you check the pipeline before you ever discuss retraining.

THE SIGNAL THAT CATCHES IT

```
positive_rate
feature p99 range
```

INCIDENT 02 · COMPUTER VISION

One ticket in twenty went to a car that was not speeding

Automated speed enforcement · Baltimore · 2012–2013 · roughly 700,000 tickets a year · found by a newspaper

THE SYMPTOM

Drivers began contesting citations that recorded speeds their vehicles could not have reached.

Each complaint was handled on its own. Nobody ever turned them into an error rate per camera.

WHAT WAS FOUND

The contractor's own review of five cameras found a 5.2% error rate — one wrong ticket in twenty — blamed on radar effects from high-walled trucks.

An independent audit went further: over 10% clearly erroneous, and another 26% questionable.

ROOT CAUSE

Per-device accuracy was never measured. Officials had been quoting an error rate of one quarter of one percent.

The human check was a formality: one officer typically reviewed 1,000 to 1,200 citations a day.

THE FIX

Erroneous tickets voided and refunded, and the entire programme taken offline within a week of the audit landing.

At 700,000 tickets of $40, a 10% error rate is $2.8 million wrongly charged.

THE LESSON

An average across eighty-three cameras will happily hide the one that has gone blind — monitor per device, against that device’s own baseline.
And count the seconds your human reviewer actually has per item before you call them a control.

THE SIGNAL THAT CATCHES IT

```
citations per camera vs baseline
review seconds per citation
```

INCIDENT 03 · RAG & ASSISTANTS

It gave illegal advice for five months in production

Government RAG assistant over 2,000+ official pages · October 2023 – March 2024 · found by journalists

THE SYMPTOM

Nothing looked wrong. The bot answered fluently and named the city as its source, and users had no way to tell a right answer from a wrong one.

There was a feedback form and a disclaimer. Neither is a measurement.

WHAT WAS FOUND

Reporters tested it against the actual law. On housing, worker rights and business rules the answers were often incomplete and sometimes dangerously wrong.

It told landlords they need not accept Section 8 vouchers — illegal in that city. Many answers carried no link at all.

ROOT CAUSE

There was no evaluation set of real questions with verified answers, so nothing ever checked whether an answer was grounded in the retrieved pages.

Five months of production traffic and not one quality number.

THE FIX

The bot stayed online with a stronger disclaimer and was finally taken down in January 2026.

Your version: a frozen golden set of high-stakes questions run nightly, with groundedness and citation rate reported per topic.

THE LESSON

A disclaimer is not a control, and a feedback form is not a metric.
If you have no golden set, your quality number is whatever a journalist decides to test on the day they test it.

THE SIGNAL THAT CATCHES IT

```
groundedness on golden set
answers with no citation
```

INCIDENT 04 · FORECASTING

The world changed in a week and the model did not

Grocery availability prediction · March 2020 · accuracy fell from about 93% to 61% · nothing was deployed

THE SYMPTOM

Item-availability predictions started failing at scale. Orders went in for products that were not on the shelf, and shoppers arrived to empty aisles.

The service was healthy throughout. Every prediction returned normally.

WHAT WAS FOUND

A model that had run at about 93% accuracy for predicting whether a product would be in stock dropped to 61% in March 2020.

No release, no bug. The inputs were valid — they were simply nothing like the training distribution.

ROOT CAUSE

Buying behaviour changed within a week. Toilet paper, flour, eggs and sanitiser went from occasional purchases to stockpiling.

Textbook concept drift: the same features now meant something completely different.

THE FIX

Reported response: lean on recent windows and treat the period as its own regime instead of trusting long history.

The general lesson teams took from 2020: a model that assumes a stable world needs a detector for when it stops being stable.

THE LESSON

This is the failure your labels are too slow to catch. Prediction drift and input drift both moved within days; accuracy needed weeks of fulfilment data to confirm what the output distribution had already said.

THE SIGNAL THAT CATCHES IT

```
predicted in-stock rate by day
ks vs frozen reference
```

FROM THE INDUSTRY

The classics, in one slide

Older, larger, and each one an ancestor of an incident you have just seen.

Zillow Offers

2021 · iBuying · shut down

Zillow bought homes based on its own valuation model. It tuned the model to bid more aggressively in a hot market, just as prices began to cool.

It took a $304M inventory write-down in Q3 2021, expected up to $569M in total, closed the business and cut about 25% of its workforce.

WHAT IT TEACHES

Concept drift in the most expensive form. The relationship between features and price changed, and the model kept acting on the old one. Nothing was broken — it was confidently wrong at scale.

Google Flu Trends

2013 · discontinued 2015

Search queries were used to predict flu prevalence. It was accurate at launch and celebrated as a landmark for big data.

It over-predicted flu in 100 of 108 weeks between 2011 and 2013, at one point estimating roughly double the CDC figure.

WHAT IT TEACHES

The model was fit once and barely refitted, while Google's own search algorithm and user behaviour kept changing underneath it. A simple projection from lagged CDC data beat it.

Knight Capital

2012 · $440M in 45 minutes

New code was deployed to seven of eight trading servers. The eighth still held dormant 2003 code bound to the same configuration flag.

At market open the eighth server traded uncontrollably. About $440M was lost in 45 minutes — more than the firm's market capitalisation.

WHAT IT TEACHES

Two lessons. Ninety-seven warning emails fired at 8:01am and nobody read them, because they were not alerts. And the rollback made it worse: reverting the seven good servers spread the bad code to all eight.

INCIDENT 03 · THE PRECEDENT

The same failure, in a tribunal

Moffatt v. Air Canada, British Columbia Civil Resolution Tribunal, February 2024.

What happened

A passenger asked Air Canada's website chatbot about bereavement fares after his grandmother died. The chatbot told him he could book now and apply for the discounted rate retroactively, within 90 days. He booked full-fare flights on that basis.

Air Canada's actual policy did not allow retroactive applications — and the correct information was published on a different page of the same website. The claim was refused. He sued.

The airline's defence

Air Canada argued that the chatbot was a separate legal entity, responsible for its own actions, and that the customer should have cross-checked another page of the website.

The tribunal rejected both arguments. It found the company owed a duty of care to ensure its representations were accurate, and that there was no reason a customer should know one part of a website is trustworthy and another is not. Air Canada was held liable for negligent misrepresentation.

You own what your model says

There is no "the AI did it" defence. Anything your assistant tells a customer is a statement by your company, with the same legal weight as your website.

This is a faithfulness failure

The chatbot invented a policy that contradicted the company's own documents. That is exactly what faithfulness scoring measures — and it is measurable before a customer finds it.

The damages are not the point

The award was a few hundred dollars. The precedent, the press and the review of every deployed assistant afterwards cost immeasurably more.

The uncomfortable question for any team shipping an assistant: if this happened to you tomorrow, could you produce the trace?

FROM OUR OWN PRACTICE · 01

Langfuse

What will the frontier model actually cost us?

Four teams · INT and Preprod only · the question everybody guesses at and nobody measures

THE PROPOSAL

The teams wanted to move off the open-source model we self-host and onto a frontier model behind an API key — for development and testing only, not production.

The assumption in the room: "it is only dev traffic, it will be small."

HOW WE MEASURED

We already had Langfuse on INT and Preprod, so we did not have to estimate anything.

We pulled token usage per team, per week — deliberately the minimum and maximum week of each month, not the average, because the average hides exactly the weeks that hurt.

WHAT WE FOUND

Four teams, one month, non-production environments only: nearly 5 billion tokens.

Developers iterating all day, plus the automated evaluation harness re-running suites on every change — the harness alone was over a fifth of it.

THE DECISION

The proposal went to management with a spreadsheet instead of an opinion.

Outcome: frontier model for a scoped set of cases, the harness pinned to the self-hosted model, and per-team token budgets with alerts — decided from numbers, not from instinct.

THE LESSON

The value of observability is not only catching failures. It is walking into a budget conversation with measured numbers instead of an estimate — and being believed.

THE SIGNAL THAT CATCHES IT

```
tokens by team, env
min/max week per month
```

FROM OUR OWN PRACTICE · THE EVIDENCE

Langfuse

The number that ended the argument

Exported straight out of Langfuse, grouped by team and environment. No estimates, no extrapolation.

llm_token_usage_INT_PREPROD_monthly.xlsx — Sheet1

	A	B	C	D	E	F
1	**Team**	**Env**	**Min week**	**Max week**	**Month tokens**	**Est. cost**
2	Product Assistant	INT	96,400,000	402,700,000	1,010,000,000	$10,100
3	Product Assistant	PREPROD	41,200,000	168,900,000	428,000,000	$4,280
4	Search & RAG	INT	88,700,000	351,400,000	902,000,000	$9,020
5	Search & RAG	PREPROD	33,500,000	142,600,000	371,000,000	$3,710
6	Agents & Automation	INT	74,900,000	318,200,000	815,000,000	$8,150
7	Agents & Automation	PREPROD	28,300,000	121,800,000	316,000,000	$3,160
8	Eval harness (automated)	INT	61,500,000	289,400,000	690,000,000	$6,900
9	Eval harness (automated)	PREPROD	38,900,000	164,200,000	400,000,000	$4,000
10	**TOTAL — non-production only**				**4,932,000,000**	**$49,320**

Cost column assumes a blended $10 per million tokens — replace it with your provider's current rate.

Min and max, never the average

A mean week would have shown about 1.2 billion and hidden the fact that a single heavy week ran four times the quiet one. Budgets are set by the peak, not the mean.

The harness was 22% of it

Automated evaluation re-runs suites on every change. It is the right thing to build — and it is a token consumer that nobody thinks to count.

This is non-production traffic

Four teams, no customers, no production load. Annualised, it is roughly $590k of spend that had been assumed to be negligible.

Before Langfuse this conversation was two people disagreeing about a guess. After it, it was a one-page spreadsheet and a decision in ten minutes.

FROM OUR OWN PRACTICE · 02

Langfuse

Retrieval quietly returned nothing — in one language

Bilingual RAG · a nightly job, an honest model, and a very confusing morning

THE SYMPTOM

After a routine overnight job, 18% of answers became "I don't have enough information to answer that."

Only Arabic questions. English was perfect. No errors anywhere.

WHAT WE FOUND

The traces showed it immediately: the retrieve span was returning zero chunks, and only for Arabic queries. The generation step was behaving correctly — it genuinely had nothing to work with.

ROOT CAUSE

The index had been rebuilt with a new embedding model. The application still encoded queries with the old one.

The two vector spaces were unrelated, and the normalisation of Arabic text differed between them.

THE FIX

Rebuilt the index with the matching encoder.

The embedding model name and normalisation settings are now stored in the index metadata, and the app refuses to start if they do not match. Alert on zero-hit rate above 5%.

THE LESSON

In a RAG system the index and the query encoder are one artifact — version them together. And always measure your retrieval and quality metrics per language, never pooled.

THE SIGNAL THAT CATCHES IT

```
retrieval_zero_hits
context_recall by lang
```

THE PATTERN

Four incidents, four lessons

Notice what is missing from this list: "the model was bad".

The model was never the root cause

A degraded device, a corpus nobody evaluated, a vendor pipeline, a world that changed. Investigate in that order — the model itself is last.

01 · 02 · 03 · 04

Nothing ever crossed a threshold

Every one of these returned 200 OK at normal latency for the whole incident. A service dashboard was green while the predictions were wrong — for weeks, or months.

all four

Every one was found from outside

A newspaper, a team of journalists, lenders, and customers staring at empty shelves. Not one team's own monitoring raised it first.

all four

The missing signal was always cheap

Citations per camera. Groundedness on fifty questions. KS on one vendor attribute. Predicted positive rate by day. None of these is hard. They were simply not collected.

all four

If you remember one thing from this chapter: none of these systems was broken in a way your uptime dashboard could see, and every one of them was found by someone outside the team.

CHAPTER 3 · SOURCES

Every incident in this chapter is public

Read at least one primary source before you tell any of these stories to someone else.

Equifax

banking · March–April 2022

The company statement on the coding issue and its consumer FAQ, plus Wall Street Journal and Register coverage from August 2022.

WHERE TO READ IT

equifax.com/newsroom — statement on recent coding issue

Baltimore speed cameras

computer vision · 2012–2014

The Baltimore Sun investigation and the URS audit it obtained, plus the city auditor's later performance audit of the programme.

WHERE TO READ IT

baltimoresun.com — speed camera audit, Jan 2014

NYC MyCity chatbot

RAG · 2023–2024

The Markup and THE CITY investigation of March 2024, the AP follow-up, and the January 2026 report on the bot being taken down.

WHERE TO READ IT

themarkup.org — NYC AI chatbot

Instacart, March 2020

forecasting · concept drift

MIT Technology Review's May 2020 reporting on pandemic model breakage, and later write-ups of the drop from about 93% to 61%.

WHERE TO READ IT

technologyreview.com — pandemic AI models

Homework: pick one, read the primary source, and write the five-part summary yourself before the next session.

CHAPTER 4

Langfuse, In Depth

Langfuse

- The data model
- Four ways to instrument
- The metadata contract
- Sessions and users
- Prompt management
- Datasets and experiments
- Scores and evaluators
- Cost analytics
- Self-hosting and privacy

THE USE CASE

What Langfuse is actually for

Langfuse

It answers three questions that Prometheus and Grafana structurally cannot answer about an LLM app.

"What exactly happened in this request?"

TRACING

Prometheus can tell you the p95 latency of your RAG endpoint. It cannot show you which five chunks were retrieved, what the model was actually asked, and what it said back. A trace can.

"Which version produced this behaviour?"

PROMPT MANAGEMENT

Prompts change more often than code and change behaviour more than code. Langfuse stores them, versions them, serves them by label, and links every trace to the version that produced it.

"Was the answer any good?"

EVALUATION

There is no label and no exception. Quality has to be attached as a score — from a judge model, a human reviewer, or a thumbs-up — and then tracked over time like any other metric.

Open source, self-hostable

The whole platform runs inside your own network. For most teams handling customer data this is the difference between approved and blocked.

Framework-agnostic

Decorators, an OpenAI drop-in wrapper, native handlers for LangChain and LlamaIndex, or plain OpenTelemetry. You are not committing to a framework.

Async by design

Events are batched on a background thread. A slow or dead Langfuse adds no latency to your user's request — you just lose observability until it returns.

DATA MODEL

Five objects. Learn these and the SDK explains itself.

Langfuse

Trace	*One end-to-end request*	The unit you search, filter and score. Carries user_id, session_id, tags, release and the top-level input and output.
Observation	*One step inside a trace*	Three kinds — SPAN for any work, GENERATION for a model call, EVENT for a moment in time. They nest, which produces the waterfall.
Session	*A group of related traces*	Use it when you model each turn as its own trace rather than as spans inside one conversation trace. Both patterns are valid — see the next slide.
Score	*A number on a trace or a step*	Numeric, boolean or categorical. From a judge, a human, or your own code. This is how quality becomes a time series.
Dataset	*Inputs plus expected outputs*	Your regression suite. Run any version of the app against it and compare runs side by side.

HOW THEY NEST

```
TRACE  conversation_4f2a
  SPAN  turn 1
    SPAN  retrieve
      SPAN  vector search
    GENERATION  answer
      SCORE  faithfulness 0.91
    EVENT  guardrail passed
  SPAN  turn 2
    SCORE  thumbs_up true
```

A score can sit on the whole trace or on one step inside it. "The conversation went wrong" and "turn 4 was unfaithful" are different claims.

INSTRUMENTATION

Four ways in — pick the least invasive one that works

Langfuse

app/instrumentation.py

```python
# 1 - decorator: any function becomes a span
@observe(name="retrieve")
def retrieve(q, k=5):
    return store.search(q, k=k)

# 2 - drop-in wrapper: model calls become generations,
#     with tokens, cost and latency captured for free
from langfuse.openai import openai

# 3 - framework callback: LangChain / LlamaIndex
from langfuse.langchain import CallbackHandler
chain.invoke(x, config={"callbacks": [CallbackHandler()]})

# 4 - OpenTelemetry: if you already ship OTel spans,
#     point the exporter at Langfuse and keep your stack

# enrich whatever is currently active
langfuse.update_current_span(metadata={"k": k})
langfuse.update_current_trace(user_id=uid, tags=["rag"])
```

Start with 2, then add 1

The wrapper alone gives you cost and token counts on day one. Add decorators only where you need to see the steps.

Name spans after the job

retrieve, rerank, guardrail_in — not helper_fn. These names become the columns you group by later.

Flush before you exit

In scripts, notebooks and serverless handlers, call langfuse.flush() or the batch dies with the process.

Never trace secrets

The input and output of every span is stored. Mask credentials and personal data at the SDK boundary, not afterwards.

THE METADATA CONTRACT

Langfuse

Traces are only useful if you can filter them

Agree these fields once, as a team. Every one of them is a question you will need to answer during an incident.

FIELD	EXAMPLE	THE INCIDENT QUESTION IT ANSWERS
user_id	u_84f21	Who is affected, and who do we apologise to? Also powers per-user cost.
session_id	chat_4f2a	Show me the whole conversation — did turn 2 poison turn 7?
release	git sha or v2.4.1	Did this start with a deploy? Compare quality before and after a release.
prompt version	linked automatically	Which prompt produced this? The fastest filter you have during an LLM incident.
environment	prod | staging | dev	Keep test traffic out of your production quality numbers. Non-negotiable.
tags	rag, arabic, tenant_x	Slice quality by feature, language and customer — where degradation actually hides.
metadata	k=5, index=v12, model	The knobs. When quality moves you need to know what the knobs were set to.

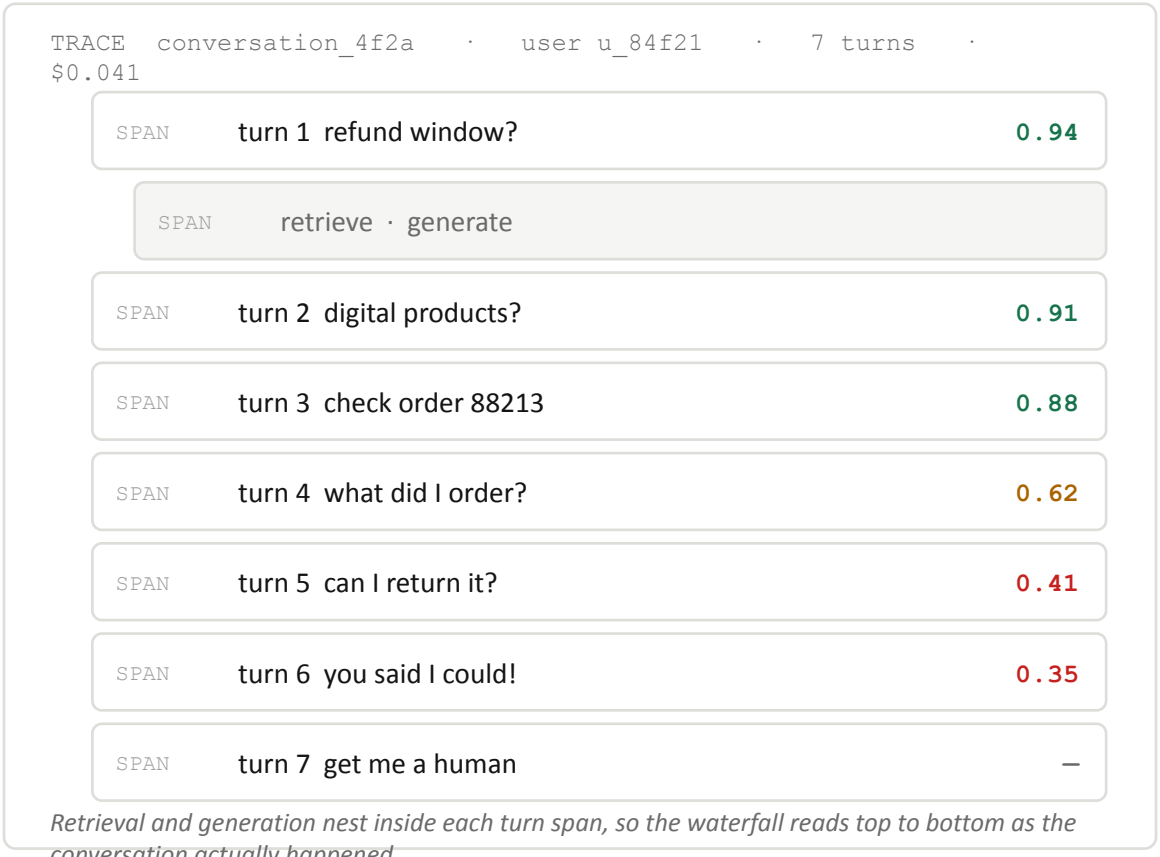
The test: take Incident 03. Could you list every answer that made a policy claim with zero retrieved chunks, in under a minute? If not, a field is missing.

MODELLING A CONVERSATION

Langfuse

One conversation, one trace — each turn a span

Technically a conversation is not required to be a single trace. In practice this is the shape you want.



Retrieval and generation nest inside each turn span, so the waterfall reads top to bottom as the conversation actually happened.

Why one trace per conversation

The whole exchange is a single object you can open, read in order and score as a unit. Cost per conversation is immediate, and the arc of a failure is visible at a glance.

The failure started at turn 4

Quality collapsed the moment the conversation needed something from an earlier turn. Turn 7 is all you would ever see if each turn were a separate, unlinked trace.

The alternative: a trace per turn, grouped by session_id

Also valid, and better when conversations are very long or unbounded, or when you need to score and search individual turns independently. You lose the single-object view.

Either way, set session_id and user_id from the first trace

Adding them later means your existing history is not searchable. This is the cheapest decision in the whole stack and the most expensive one to postpone.

THE PRODUCT

Langfuse

Four screens you will actually live in

Everything in this chapter surfaces in one of these. Know which screen answers which question.

Tracing & sessions

What happened?

- Filter by user, session, tag, release, prompt version
- Open one request and read every step in order
- Replay a whole conversation

Where you go during an incident.

Prompt management

What did we ask?

- Every version, immutable, with model and temperature
- Labels — production, canary — as movable pointers
- Edit and test in the UI, roll back without a deploy

Where a two-day outage becomes a two-minute fix.

Scores dashboard

Was it any good?

- Judge, human and programmatic scores in one place
- Trend over time, grouped by prompt version or tag
- Drill from a score straight into the failing traces

Where quality stops being a feeling.

Cost & token usage

What did it cost?

- Tokens and cost per trace, user, tag and model
- Input vs output token split
- Export by team and environment — see the token-budget case in chapter 3

Where you get numbers for management.

Plus datasets and experiments, which is where the offline loop lives — covered two slides from now.

PROMPT MANAGEMENT

Langfuse

Prompts are config — version, label, link, roll back

Incident 03 ran wrong for five months. Versions, labels and a gate are how that becomes a two-minute rollback instead of a quarter.

app/prompts.py

```python
prompt = langfuse.get_prompt("rag-system",
                             label="production",
                             cache_ttl_seconds=60)

compiled = prompt.compile(tone="concise", language="Arabic")
resp = openai.chat.completions.create(
    model=prompt.config["model"],      # config is versioned too
    temperature=prompt.config["temperature"],
    messages=[{"role": "system", "content": compiled}, ...],
    langfuse_prompt=prompt,    # links trace -> exact version
)
```

The promotion path that prevents Friday incidents

edit in the UI → run against the dataset → CI checks faithfulness and cost → label as canary for 10% of traffic → compare scores → move the production label

Versions and labels

Every save is an immutable version. Labels — production, canary — are movable pointers, so promotion and rollback are a label move, not a deploy.

Config travels with it

Model name, temperature and max_tokens live on the version. Rolling back the prompt rolls back the settings it was tuned with.

Linked to every trace

"Quality dropped Tuesday" becomes "v14 cost us 9 points of faithfulness". Any score can be grouped by prompt version.

Cached, and fails open

The SDK caches locally and refreshes in the background. If Langfuse is unreachable your app keeps serving the last good prompt.

DATASETS & EXPERIMENTS

Langfuse

Turn production failures into a regression suite

The loop that stops you fixing the same bug twice.

1 · Capture

A user thumbs-downs an answer, or a judge score comes back low. That trace is now evidence.

→

2 · Promote

Add the trace to a dataset — one click in the UI, or by API. The input, plus the expected output if you know it.

→

3 · Experiment

Run a new prompt, model or retrieval setting against the whole dataset as a named run.

→

4 · Compare

Scores per run, side by side, per item. You see what improved and — more importantly — what regressed.

evals/run_experiment.py

```python
dataset = langfuse.get_dataset("support-golden-v3")

for item in dataset.items:
    with item.run(run_name="prompt-v15",
                  run_metadata={"model": "gpt-4o-mini"}) as root:
        output = my_rag_app(item.input)
        root.score_trace(name="faithfulness",
                         value=judge(output, item.expected_output))
```

What belongs in the dataset

- Every reported bad answer, permanently
- The hard cases: multi-hop, ambiguous, out of scope
- Questions with no answer in your documents — the model must say so
- Both languages, if you serve both
- A few boring easy ones, to catch catastrophic regressions

50 well-chosen items beat 5,000 scraped ones. The dataset is a design artifact, not a dump.

SCORES

Three ways quality gets a number

Langfuse

All three write to the same place, so all three end up on the same chart.

Programmatic

cheap · exact · narrow

- Did it return valid JSON?
- Did retrieval return zero chunks?
- Did it cite a source?
- Was it inside the length limit?

Run on 100% of traffic. No excuse not to.

LLM-as-a-judge

scalable · noisy · needs calibration

- Faithfulness to the context
- Relevance to the question
- Tone and policy compliance
- RAGAS metrics land here

Run on 1–5% of live traffic, 100% of your dataset.

Human

expensive · the ground truth

- Thumbs up / down from users
- An annotation queue for reviewers
- Expert review of the hard cases

The only source that tells you whether the judge is right.

any source, one API

```
01 langfuse.create_score(trace_id=tid, name="faithfulness", value=0.91, comment="2 of 23 claims unsupported")
```

Langfuse can also run judge evaluators for you on a sample of live traffic — configured in the UI, with no job for you to maintain.

COST & LATENCY ANALYTICS

Langfuse

The numbers behind the token-budget decision

Cost per trace	Total spend hides everything. Cost per trace is the number that moves when someone changes k, a prompt or a model.	**Alert above 2× the 7-day baseline**
Input vs output tokens	Input growing while output stays flat means you are sending more, not getting more. That is almost always a bug.	**Watch the ratio, not the totals**
Cost per user / feature	One tenant or one feature is usually responsible for most of the bill. Tags make this a single group-by.	**Review weekly, not monthly**
Time to first token	What users actually feel. A 6-second answer that starts in 400 ms feels fast; the same answer in one block does not.	**Alert on p95 above 2 s**
Duration ÷ output tokens	Total duration is meaningless alone — longer answers take longer. Normalise before comparing models or versions.	**Compare across prompt versions**

Langfuse prices each generation from the model name and token counts, so cost is available per trace, per user and per tag with no extra work.

RUNNING IT IN PRODUCTION

Langfuse

Self-hosting, privacy, and joining the rest of your stack

What it runs on

- langfuse-web — UI and API
- langfuse-worker — async ingestion
- PostgreSQL — metadata
- ClickHouse — traces and observations
- Redis — queue and cache
- S3 / MinIO — large payloads

Docker Compose for a lab, Helm for a cluster.



Privacy, before it is a problem

- Mask personal data at the SDK boundary
- Traces store full input and output — assume that in your review
- Sample production traces, but keep 100% of failures
- Set retention per project
- Separate prod and staging projects

Masking after ingestion is not masking.

Joining your existing stack

- Export aggregate metrics to Prometheus and alert in Grafana with everything else
- Push scores back as metrics so quality sits next to latency
- Run dataset experiments in CI on every prompt or model change

One dashboard, one on-call, one runbook.



CHAPTER 5

RAGAS, In Depth

RAGAS

- The mental model
- The metric map
- How faithfulness is computed
- How relevancy is computed
- Precision and recall
- Building a test set
- Choosing and calibrating the judge
- Running it in CI
- Reading the scores

THE MENTAL MODEL

RAGAS

RAGAS turns one answer into a set of numbers

There is no magic and no trained model. Every metric is an LLM being asked a very specific, very small question.

A sample

```
user_input
retrieved_contexts
response
reference (optional)
```

One row = one question your system answered.

→

A metric

```
faithfulness
answer_relevancy
context_precision
context_recall
```

Each one decomposes the sample into small yes/no judgements.

→

A judge LLM

```
gpt-4o-mini
or any model you choose
```

Answers those small questions. This is where the cost and the noise come from.

→

A score

```
0.0  →  1.0
```

Higher is better for every RAGAS metric. Almost all are a ratio of things that passed.

It is a judge, not a measurement

Every score is an opinion produced by a language model. Treat it as a noisy instrument you must calibrate — not as ground truth.

It splits blame between the two halves

A bad answer is either retrieval's fault or generation's fault. The metric set is designed to tell you which — that is its real value.

Absolute numbers lie, deltas do not

Nobody can tell you what a "good" faithfulness is. What matters is the change against your own baseline on your own dataset.

THE METRIC MAP

RAGAS

Which metric blames which component

Pick metrics by the question you need answered, not by how many you can fit on a dashboard.

METRIC	BLAMES	THE QUESTION IT ASKS	NEEDS A REFERENCE ANSWER?
Faithfulness	Generation	Is every claim in the answer supported by the retrieved text?	No — reference-free
Response relevancy	Generation	Does the answer address the question that was asked?	No — reference-free
Context precision	Retrieval	Are the useful chunks ranked near the top?	No, if you judge against the answer
Context recall	Retrieval	Did retrieval bring back everything needed to answer?	Yes — this is the expensive one
Factual correctness	End to end	Does the answer match the known-correct answer?	Yes
Noise sensitivity	Generation	Does an irrelevant chunk push the model into errors?	Yes

Start with three

Faithfulness catches hallucination · Response relevancy catches waffle · Context precision catches bad retrieval. Those three need no ground truth, so you can run them on live production traffic today. Add context recall when you have a labelled dataset.

UNDER THE HOOD · 1

RAGAS

Faithfulness: claim by claim

The anti-hallucination metric. Understand this one and the rest follow the same shape.

1

Break the answer into atomic claims

The judge splits the response into standalone factual statements. "Refunds take 14 days and cost 20 EGP" becomes two claims.

2

Check each claim against the context

For every claim: can it be inferred from the retrieved chunks? Yes or no. The question is never "is it true in the world".

3

Score = supported ÷ total

Three of four claims supported gives 0.75. That is the entire formula.

WORKED EXAMPLE

Context: "Returns are accepted within 14 days of delivery. The customer pays return shipping."

Answer: "You can return it within 14 days. Shipping is on you, and you also get a free replacement."

✓ Returns accepted within 14 days

✓ Customer pays return shipping

✗ A free replacement is provided

```
faithfulness = 2 / 3 = 0.67
```

The model invented a replacement policy. One sentence, and the score drops by a third — which is exactly the sensitivity you want.

Faithfulness says nothing about whether your documents are correct. A confidently wrong document produces a perfectly faithful wrong answer.

UNDER THE HOOD · 2

RAGAS

Response relevancy: ask the question backwards

A clever trick — instead of judging the answer, it reconstructs the question from it.

1

Generate questions from the answer

The judge reads only the answer and writes n questions that this answer would perfectly respond to.

2

Embed and compare

Each generated question is embedded and compared to the real user question with cosine similarity.

3

Score = mean similarity

If the answer was on topic, the reconstructed questions look like the original. If it wandered, they do not.

WHAT A LOW SCORE ACTUALLY MEANS

The answer is incomplete

It addressed half the question. The reconstructed questions are narrower than the original.

The answer hedged

"It depends on several factors…" reconstructs into vague questions that match nothing.

The answer drifted

It answered a related but different question — very common when retrieval returns near-miss chunks.

It refused

A refusal is not relevant to the question. Expect low relevancy on legitimate refusals, and exclude them before you panic.

Relevancy does not check whether the answer is correct — only whether it is on topic. Faithfulness and relevancy are both required; neither is sufficient.

UNDER THE HOOD · 3

RAGAS

Context precision and recall: grading the retriever

These two are the difference between "the model hallucinated" and "the model never had a chance".

Context precision

"Of the chunks I retrieved, how many were useful — and were they at the top?"

The judge marks each retrieved chunk as relevant or not, then computes a rank-weighted average — a useful chunk at position 1 counts for more than the same chunk at position 8.

#1	Return window policy	relevant
#2	Shipping fees table	relevant
#3	Company history page	noise
#4	Careers page	noise

Two useful chunks, both ranked first → high precision. Same two at #7 and #8 → much lower.

Context recall

"Of everything needed to answer, how much did retrieval actually find?"

The judge breaks the reference answer into claims and checks each one against the retrieved chunks. Score = claims found ÷ claims needed. This one needs ground truth, which is why it only runs on your dataset and never on live traffic.

HOW TO READ THE PAIR

Low recall, high precision

You find good chunks but miss things. Chunk smaller, raise k, or fix your embeddings.

High recall, low precision

You find everything plus a lot of noise. Add a reranker or lower k — noise costs money and confuses the model.

High both, low faithfulness

Retrieval is fine. The generation step is the problem — look at the prompt.

THE TEST SET

RAGAS

Your evaluation is only as good as your questions

This is where most teams fail — not at the metrics, at the dataset.

Mine production

The best source. Every thumbs-down, every escalation, every low judge score becomes a permanent test case. Real users ask questions you would never invent.

Generate synthetically

RAGAS can build a test set from your own documents — simple lookups, multi-hop questions that need two chunks, and questions written in different personas or styles.

Write the adversarial ones by hand

Questions with no answer in your corpus, ambiguous questions, questions that mix two policies, and questions in each language you serve.

evals/build_testset.py

```python
from ragas.testset import TestsetGenerator

generator = TestsetGenerator.from_langchain(llm, embeddings)
testset = generator.generate_with_langchain_docs(docs, testset_size=60)
```

Rules that save you later

- Always review generated questions by hand — some will be nonsense
- Freeze the set and version it; a moving test set makes every comparison meaningless
- Keep it balanced by language and by question type

A 60-question set you trust beats a 2,000-question set you have never read.

THE JUDGE

RAGAS

Your scores are only as trustworthy as your judge

This is the step teams skip, and it is the reason so many people quietly stop believing their eval numbers.

Calibrate against humans, once

Hand-label 50–100 examples. Measure agreement between your judge and your humans. Below roughly 80% agreement, the judge is a random number generator with good grammar.

Pin the model and the version

A judge model upgrade silently shifts every score you have. Pin it, record it in the run metadata, and re-baseline deliberately when you change it.

Temperature 0, always

You want the same answer for the same input. Any randomness in the judge shows up as drift you will spend a day investigating.

Watch the cost

Faithfulness makes several LLM calls per sample. A 500-item set is not free, and it is why you sample live traffic instead of judging all of it.

If you work in Arabic, test this before you trust it

Judge models are markedly weaker at claim decomposition in Arabic than in English, and dialect makes it worse. Run your calibration set in both languages, report scores per language, and never compare an Arabic score against an English one.

RUNNING IT

Three places RAGAS belongs

Langfuse RAGAS

evals/run_eval.py

```python
from ragas import evaluate, EvaluationDataset
from ragas.metrics import (Faithfulness, ResponseRelevancy,
                           LLMContextPrecisionWithoutReference)

ds = EvaluationDataset.from_list(samples)    # from prod traces
result = evaluate(ds, metrics=[Faithfulness(),
                               ResponseRelevancy(),
                               LLMContextPrecisionWithoutReference()])

if result["faithfulness"] < 0.85:
    raise SystemExit("quality gate failed")    # fails the build
```

In CI, as a gate

Every prompt change, model change or retrieval change runs against the frozen dataset. Below threshold, the build fails. This is what would have stopped Incident 03.

Nightly, on sampled production

Pull yesterday's traces from Langfuse, score 1–5% of them, write the scores back. Now quality is a trend line next to latency and cost.

Before promoting a prompt

Run the candidate against the dataset, compare per-item against the current production version, and look at what regressed — not just the average.

RAGAS + Langfuse together

Langfuse gives you the samples (question, contexts, answer already sit in the trace) and takes back the scores. RAGAS computes them. Traces in, scores out — and quality becomes a filterable, groupable, alertable dimension next to prompt version, tenant and language.

READING THE SCORES

RAGAS

Six ways to fool yourself with an eval number

Chasing an absolute number

There is no universal "good" faithfulness. 0.82 on a hard legal corpus may be excellent; 0.95 on FAQs may be poor. Compare only against your own baseline.

Reporting the mean only

One mean hides everything. Look at the distribution and at the worst items — a bimodal 0.85 (half perfect, half broken) is a different system from a uniform 0.85.

Evaluating on a moving test set

If the dataset changed between runs, the comparison is meaningless. Freeze and version it like code.

Ignoring refusals

Legitimate "I don't know" answers score low on relevancy. Separate them out, or you will optimise your system into confidently answering things it should refuse.

Pooling languages or tenants

A stable overall score can hide one language collapsing. Always report per segment — this is how our Arabic zero-hit case stayed invisible.

Trusting an uncalibrated judge

Without human agreement numbers, you are optimising against an opinion you have never checked. Calibrate before you make a decision on the score.

The only question an eval number has to answer: is this version better or worse than the one currently in production, on the cases I care about?

CHAPTER 6

Interview Preparation

- Twelve questions you will be asked
- What each one is really testing
- A strong answer for each
- What weakens an answer
- How to tell an incident story

BEFORE THE QUESTIONS

How to tell a production story in five sentences

Almost every MLOps interview turns into "tell me about something that broke". Have three of these ready.

Context	10 sec	*"We ran a bilingual RAG assistant over policy documents, rebuilt nightly."*	Scale and stakes, nothing else. Do not describe the architecture.
Signal	20 sec	*"One morning 18% of answers said 'not enough information' — Arabic only, no alert."*	A number and a time. This is the sentence that shows you actually operate systems.
Action	40 sec	*"I checked deploys first, then the traces — retrieval returned zero chunks."*	Your reasoning in order. Say what you ruled out, not just what you found.
Outcome	20 sec	*"The nightly job had rebuilt the index with a different encoder. A morning of answers was wrong."*	Include the damage honestly. Owning the cost reads as senior, not weak.
Lesson	20 sec	*"Index and encoder are one versioned artifact now, and we alert on zero hits per language."*	A systems fix, not "we were more careful". This is the sentence they remember.

The most common mistake: spending four minutes on the architecture and thirty seconds on what you actually did.

FUNDAMENTALS

Monitoring without labels

"How do you know a model is degrading if you have no labels?"

WHAT THEY ARE REALLY TESTING

Whether you have actually run something in production, or only read about drift.

A STRONG ANSWER

- Name the tiers: service health now, input and output drift within hours, true accuracy only when labels arrive — sometimes never.
- Lead with prediction drift: the output distribution is free, needs no labels, and moves before anything else you can measure.
- Add input health — null rates, schema failures, range checks — because most degradation is an upstream bug.
- Close with a proxy: escalation rate, override rate, complaints, conversion. The business always has a delayed signal.

WHAT WEAKENS IT

Answering only "we monitor accuracy". If labels arrive in two weeks, you have just said you would find out in two weeks.

"What is the difference between data drift and concept drift?"

WHAT THEY ARE REALLY TESTING

Whether you understand that these need completely different responses.

A STRONG ANSWER

- Data drift: the inputs moved, the rule is unchanged — P(X) changes. Often harmless.
- Concept drift: the same input now means something different — P(Y|X) changes. Always harmful.
- Give one concrete example of each. Interviewers remember the example, not the notation.
- The key point: data drift is easy to detect and often ignorable; concept drift is hard to detect and never ignorable — which is why you need labels or a good proxy.

WHAT WEAKENS IT

Reciting the formulas with no example, or implying that any detected drift means you should retrain.

OPERATIONS

Incidents and alerting

"The dashboard is all green but the business says the model is bad. What now?"

WHAT THEY ARE REALLY TESTING

Your debugging order. This is the most common real-world scenario in the job.

A STRONG ANSWER

- Say the quiet part first: green dashboards measure the service, not the predictions. That gap is the whole problem.
- Segment before anything else. A flat global metric routinely hides one region, one language or one tenant collapsing.
- Check the threshold and the class balance — a shifted positive rate breaks a cut-off tuned months ago.
- Then check the data path: nulls, schema, units, and whether preprocessing at serving still matches training.

WHAT WEAKENS IT

Jumping straight to "we should retrain". It signals you have never found the real cause of a production problem.

"What would you alert on — and what would you deliberately not alert on?"

WHAT THEY ARE REALLY TESTING

Whether you have ever been on call. Juniors alert on everything.

A STRONG ANSWER

- Page on symptoms users feel: errors, latency past the agreed limit, no predictions being served, target down.
- Warn on causes: drift, GPU memory creeping, a growing queue, a feature that started returning nulls.
- Explain the "for" window — drift is slow, so a 10-minute window costs nothing and removes almost all false alarms.
- Say explicitly what you would not page on: a single drifted feature, a one-off latency spike, anything with no runbook.

WHAT WEAKENS IT

Listing twenty metrics you would alert on. That is the answer that gets people paged at 3 a.m. for nothing.

TOOLING

Prometheus and retraining decisions

"Why do you always wrap a counter in rate()?"

WHAT THEY ARE REALLY TESTING

A small, sharp question that instantly separates people who have used Prometheus from people who have read about it.

A STRONG ANSWER

- A counter only goes up and resets to zero when the process restarts. Its raw value is an artifact of uptime, not of behaviour.
- rate() gives per-second change over a window and handles the resets for you.
- Bonus point: mention cardinality unprompted — never label with user_id or request_id, because every distinct value is a new time series.
- If they push further: histogram buckets must be summed before histogram_quantile, otherwise you are averaging percentiles, which is meaningless.

WHAT WEAKENS IT

Saying "it smooths the graph". That is a side effect, not the reason.

"How do you decide when to retrain?"

WHAT THEY ARE REALLY TESTING

Whether you would build an automation that quietly makes things worse.

A STRONG ANSWER

- Start by rejecting the premise: drift detected is not retrain needed. Say that out loud.
- Give the gates — sustained over days not one window, upstream data verified healthy, enough new labels to learn from, and a cooldown since the last retrain.
- Validate the challenger on a held-out normal period, never on a random split of the anomalous window that triggered it.
- Mention the seasonal case — a holiday or a campaign is drift you ride out, not drift you learn.

WHAT WEAKENS IT

Answering "when drift exceeds the threshold, we retrain automatically". That is a pipeline that will one day train on a broken upstream feed.

LLM SYSTEMS

Evaluating and observing LLM applications

"How do you evaluate an LLM application with no ground truth?"

WHAT THEY ARE REALLY TESTING

Whether you can reason about quality when there is nothing to compare against.

A STRONG ANSWER

- Split it into three sources: programmatic checks on 100% of traffic, an LLM judge on a sample, and humans on the hard cases.
- Name reference-free metrics specifically — faithfulness, response relevancy, context precision all work without ground truth.
- Explain the split: faithfulness blames generation, context precision blames retrieval. Knowing which half is broken is the point.
- Add the feedback loop: every reported bad answer becomes a permanent dataset item, so the same failure cannot come back silently.

WHAT WEAKENS IT

Saying "we use RAGAS" and stopping. Name the metrics, and say what each one blames.

"Explain faithfulness. How is it actually computed?"

WHAT THEY ARE REALLY TESTING

Whether you understand your tools or just import them.

A STRONG ANSWER

- Three steps: decompose the answer into atomic claims, check each claim against the retrieved context, score = supported ÷ total.
- Stress what it does not do: it checks support by the context, not truth in the world. A wrong document gives a faithful wrong answer.
- Note that it is reference-free, which is why it can run on live production traffic.
- Mention the cost — several judge calls per sample, which is why you sample rather than score everything.

WHAT WEAKENS IT

"It measures hallucination." True but shallow — the follow-up question is always "how", and that is the real question.

LLM SYSTEMS

Cost and system design

"An LLM feature's cost tripled with no deploy. How do you find out why?"

WHAT THEY ARE REALLY TESTING

Whether you treat cost as an engineering signal or as somebody else's problem.

A STRONG ANSWER

- Go straight to cost per trace, not total spend. Total spend tells you that something happened; per trace tells you what.
- Split input tokens from output tokens. Input rising while output is flat means you are sending more, which is almost always a config change.
- The usual suspects: retrieval k raised during an experiment and never reverted, full conversation history replayed each turn, a retry loop, a long system prompt on a cheap task.
- Segment by feature, tenant and prompt version — one of them is almost always responsible.

WHAT WEAKENS IT

Suggesting "switch to a cheaper model" first. That is a decision, not an investigation — and it may not be the problem at all.

"Design observability for a RAG system from scratch. Where do you start?"

WHAT THEY ARE REALLY TESTING

Seniority. They want to hear priorities and trade-offs, not a tool list.

A STRONG ANSWER

- Start with the failure modes, not the tools: retrieval returns nothing, retrieval returns noise, the model ignores good context, cost runs away, one language breaks.
- Then map each one to a signal — zero-hit rate, context precision, faithfulness, cost per trace, and everything reported per language.
- Layer it: traces for individual debugging, aggregate metrics for alerting, scores for quality trends.
- Name the metadata contract — user, session, release, prompt version, environment, tags — because without it none of it is filterable.

WHAT WEAKENS IT

Opening with "I would use Langfuse and Prometheus". Lead with the failure modes; the tools are the easy part.

JUDGEMENT

The questions that separate mid from senior

"How do you know your LLM-as-a-judge can be trusted?"

WHAT THEY ARE REALLY TESTING

Intellectual honesty. Most candidates have never thought about this.

A STRONG ANSWER

- Say the uncomfortable thing: an uncalibrated judge is an opinion, and optimising against it can make the product worse.
- Calibrate — hand-label 50–100 examples and measure agreement. Below roughly 80%, do not make decisions from the score.
- Control the variables: temperature 0, a pinned model version, and the version recorded in the run metadata.
- Name the known biases — judges prefer longer answers and tend to prefer their own model family.

WHAT WEAKENS IT

Treating judge scores as ground truth. The moment you say "faithfulness is 0.91 so we are fine", a senior interviewer stops listening.

"Tell me about a time your monitoring failed to catch something."

WHAT THEY ARE REALLY TESTING

Whether you can be honest about a gap and turn it into a systems improvement.

A STRONG ANSWER

- Pick a real gap — a gradual decline that never crossed a threshold is the strongest kind of story.
- Be specific about time to detect: "it took nine days, and finance found it, not us."
- Explain why the existing monitoring was structurally blind, not just unlucky. Static thresholds cannot catch slow trends.
- Say what you added — the exact signal, the exact alert, and the baseline you compare against now.

WHAT WEAKENS IT

Choosing a story where nothing was really your responsibility, or ending without a concrete systems change.

RUNBOOK

"The model degraded" — what do I actually do?

Write this before you need it. A runbook that requires thinking is a runbook that fails at 3 a.m.

00	**Is it the model at all?**	≤ 5 min	Error rate, latency, pod restarts, GPU and CPU saturation, upstream job status. A "degraded model" is usually a broken dependency.
01	**Check the data pipeline**	≤ 10 min	Null rates, schema tests, row counts, last successful upstream run. Compare feature statistics to yesterday. Look for a unit or encoding change.
02	**Check the deploy timeline**	≤ 5 min	Did a model, prompt label, feature or preprocessing release land inside the window? Roll back first, investigate second.
03	**Localize it**	≤ 20 min	Which features, which segments, which cameras, which tenants? Global and one-segment degradation need different responses.
04	**Check real quality**	≤ 30 min	Any labels, even partial or late? Any judge scores or user feedback? Drift without a quality loss is not an incident.
05	**Decide and write it down**	same day	Do nothing (seasonal), fix upstream, recalibrate the threshold, roll back, or retrain. Record the decision and the evidence.

Put this in README.md with the exact dashboard links, the exact SQL, and the exact rollback command.

ANTI-PATTERNS

Six ways monitoring projects fail

Alerting on everything

Every feature gets a page. Two weeks later the channel is muted and nobody sees the real one.

Too many label values

One request_id label melts Prometheus. Ask about cardinality in code review.

Rolling the baseline forward

Comparing this week to last week makes gradual drift mathematically invisible. Keep a frozen reference from training.

Retraining with no gate

Drift caused by a broken upstream job becomes a model trained on broken data — in production.

Watching inputs, not outputs

Concept drift leaves the features untouched. Prediction drift is the cheapest signal, and the one most teams do not collect.

Only looking at the average

Global accuracy is flat while one segment collapses. Always keep a segment dimension.

And the two LLM-specific ones: shipping an LLM feature with no evaluation, and shipping it with no view of what it costs.

WRAP-UP

Where to go next

DOCS

- prometheus.io/docs/practices
- grafana.com/docs
- docs.evidentlyai.com — pin 0.7.x
- langfuse.com/docs
- docs.ragas.io
- docs.nvidia.com/dcgm

READING

- Google — Rules of ML (rules 28–35)
- Google SRE Book — chapter 6, Monitoring
- Sculley et al. — Hidden Technical Debt in ML
- Gama et al. — Concept Drift Adaptation
- Zheng et al. — Judging LLM-as-a-Judge

PRACTICE THIS WEEK

- Instrument your service, then break it on purpose and see which alert fires first
- Calibrate a judge: hand-label 50 answers and measure agreement
- Write out one incident in the five-sentence format
- Turn your three worst LLM answers into a dataset

NEXT — SESSION 5 · PHASE 5

Optimization & Efficient Deployment

Pruning, quantization, distillation — and deploying to GPU, CPU and edge.



THREE THINGS TO REMEMBER

01

Models fail silently.

Your normal monitoring is blind to a model that returns 200 OK and the wrong answer. What you build in this session is the only detector you have.

02

Measure the whole system.

GPU, CPU, memory and queue depth explain most "model" incidents. Instrument the machine, the server and the predictions together, on one timeline.

03

An alert without a runbook is noise.

The value of this entire stack is realised in the ten minutes after it fires. Write down what to do before you need it.

Questions — then we build.

Aya · MLOps MENA Community





Evidently

Langfuse

CHAPTER A

Appendix — Extra Practice

- Six composite practice cases
- Twelve rapid-fire questions
- Not covered live — use it as homework
- Answers included

MORE INCIDENTS

Two versions serving · a model poisoning its own training data

PRACTICE CASE A1

recommendations · found by a customer complaint

Half the traffic was still hitting the old model

SYMPTOM

Two customers with identical profiles got different recommendations. Latency was bimodal — two clear clusters on the histogram. The dashboard averaged them into one healthy-looking line.

ROOT CAUSE

A rollback scaled the new deployment down but never to zero. Three canary replicas kept serving v4 for eleven days. No metric tracked which versions were live.

THE LESSON

Always label metrics with model version and alert when more than one version is serving unexpectedly.

```
count(count by (version)(up)) > 1
```

PRACTICE CASE A2

fraud · a slow decline over four retrain cycles

Every retrain made fraud recall slightly worse

SYMPTOM

Recall dropped a couple of points every retrain cycle. Each individual model validated fine. Over six months the model became measurably worse at catching the fraud it used to catch.

ROOT CAUSE

Blocked transactions never completed, so they never appeared as labelled fraud in training data. The model was progressively erasing the evidence for its own best patterns.

THE LESSON

If your model changes the world it learns from, hold out a random control slice that bypasses it — that slice is your only unbiased data.

```
recall by cohort · control slice size
```

MORE INCIDENTS

A timezone in the feature store · a disk nobody watched

PRACTICE CASE A3

transaction risk · worst between midnight and 3 a.m.

The "last 24 hours" feature was off by three hours

SYMPTOM

False positives spiked overnight and settled by morning. Daytime performance was normal, so weekly averages looked fine and nobody investigated for a month.

ROOT CAUSE

Training computed rolling windows in UTC; the serving path used Africa/Cairo local time. Every count-in-last-24h feature was shifted, and the shift mattered most across the date boundary.

THE LESSON

Time windows are features. Assert on them, test at midnight and at daylight-saving boundaries, and store the timezone in the feature definition.

```
accuracy by hour of day
```

PRACTICE CASE A4

inference service · three days of confused debugging

Intermittent 503s every few hours, with no pattern

SYMPTOM

The service returned 503s in short bursts every few hours. Restarting fixed it, so it was written off as flakiness twice before anyone looked properly.

ROOT CAUSE

The pod disk filled with rotated logs and cached model artifacts. Kubernetes evicted the pod, it restarted, reloaded the model, filled up again. A perfect loop.

THE LESSON

Alert on predicted time-to-full, not on a static disk percentage — 85% full tells you nothing about how fast you are getting there.

```
predict_linear(disk_free[6h], 4*3600) < 0
```

MORE INCIDENTS

A library upgrade · a feedback signal that went quiet

PRACTICE CASE A5

document classification · after a routine dependency bump

Only long Arabic documents got worse

SYMPTOM

Accuracy on short documents was unchanged. On documents over roughly 800 tokens it collapsed. Because long documents were 9% of traffic, the overall metric barely moved.

ROOT CAUSE

A minor version bump changed the tokenizer's default truncation behaviour. Long inputs were silently cut, and Arabic text hit the limit sooner than the English test cases did.

THE LESSON

Pin every version, and always evaluate on the long tail — input length, rare classes, each language — not just on the average case.

```
accuracy by input length bucket
```

PRACTICE CASE A6

LLM assistant · the most dangerous incident on this list

Complaints dropped, so we assumed quality improved

SYMPTOM

Thumbs-down feedback fell to almost nothing over a week. The team celebrated. Quality had in fact stayed flat, and one segment had quietly got worse.

ROOT CAUSE

A frontend release broke the feedback widget. It rendered but never submitted. The signal did not degrade — it went to exactly zero, which nothing was watching for.

THE LESSON

Monitor your monitoring. A signal going to zero is an incident, not good news — alert on the absence of data, not only on bad values.

```
rate(feedback_total[6h]) == 0
```

RAPID-FIRE INTERVIEW QUESTIONS

Short questions, short answers — 1 of 3

The kind asked in a phone screen, where you have thirty seconds each.

"What is the first thing you check when accuracy drops?"

The data pipeline, not the model. Nulls, schema, units, upstream job status, and whether a deploy landed in the window. Most degradation is an upstream change.

"How do you monitor a model whose labels arrive in 30 days?"

Tier it. Service health now, input and prediction drift within hours, real accuracy when labels land. Prediction drift is the workhorse — free, and it moves first.

"What is cardinality and why should I care?"

The number of distinct label combinations. Each one is a separate time series. An unbounded label like user_id creates millions and will take Prometheus down.

"Why report p95 rather than the average latency?"

The average hides the tail, and the tail is where users churn. One request in twenty being terrible is a real problem that a healthy mean will never show you.

RAPID-FIRE INTERVIEW QUESTIONS

Short questions, short answers — 2 of 3

"What is training/serving skew and how do you detect it?"

The two paths compute features differently. Detect it by logging the exact vectors the service used and replaying them through the offline pipeline — if the scores match, the model is fine and the inputs are not.

"Why not alert on every drifted feature?"

With enough rows, a statistical test flags shifts too small to matter. You would page someone nightly for noise, the channel gets muted, and then the real alert is missed too.

"What is the difference between a trace and a span?"

A trace is one end-to-end request. A span is one step inside it — retrieval, a tool call, a guardrail. A generation is a span for a model call, carrying tokens and cost.

"Name a reference-free and a reference-based metric."

Faithfulness is reference-free — it checks the answer against the retrieved context, so it runs on live traffic. Context recall is reference-based — it needs a known-correct answer, so it only runs on your dataset.

RAPID-FIRE INTERVIEW QUESTIONS

Short questions, short answers — 3 of 3

"Faithfulness is 0.95 but users still complain. Why?"

Faithfulness only checks the answer against the retrieved text. If retrieval brought back the wrong documents, or your documents are outdated, the answer is faithfully wrong. Check context precision and recall next.

"How would you catch a cost spike before finance does?"

Track cost per trace, not total spend, and split input from output tokens. Alert when cost per trace exceeds twice the 7-day baseline, and report it per feature and per tenant.

"Utilization or saturation — which do you alert on?"

Saturation. High utilization is what you paid for. Queue depth and wait time are what users actually feel, and a GPU can be 100% utilized while barely computing anything.

"Your judge model was upgraded. What do you do?"

Stop comparing across the change. Re-run the calibration set, re-baseline, and record the judge version in run metadata. Scores from different judge versions are not the same measurement.

Four documented incidents, six practice cases and twenty-four questions across the deck. That is enough material for a study group to run on for a month.

Home work

- Provider drift, Prompt drift, trajectory drift, evaluator drift, search for any other drifts
- if you are a junior, fresh, you just need to understand the drift meanings, what to advice the devops team to focus on monitoring ?
- For a junior you need to easily debug the system error
- if you are a senior Study on your own the setup of prometheus and grafana, or check the next session with Eng Abdallah, and Eng Mahmoud
- if you are senior please deep dive into the mentioned tests at slide 9 (KS, Chi-square, JS) and try ot by yourself with an example
-
-
-